Predicting Used-Car Prices in Sri Lanka with Ensemble Machine Learning

Applied Machine Learning — Assignment Report

Name: Lakshan B.D. Index: 269136C Date: 23 October 2026

Live demo: https://crazy-diya-ml-assignment-269136c-app-8yzt7l.streamlit.app/

Code and app: https://github.com/crazy-diya/ML_Assignment-269136C

1. Problem and data

Sri Lanka's car market is unusual: vehicle imports were restricted from 2020, pushing used-car prices up and making them opaque; many cars are "reconditioned" Japanese imports; and hybrids and EVs are growing quickly. Buyers and sellers need a fair-price estimate. This project predicts the asking price (LKR) of a car from its specification — a tabular regression problem.

Data were collected from ikman.lk, Sri Lanka's largest classifieds site, with a polite scraper (robots.txt respected, 1.5 s delay between requests). The dataset has 3,000 advertisements and 10 usable fields: brand, model, year, condition (Brand New / Used / Reconditioned / Import), transmission, body type, fuel type, engine capacity, mileage and location. Prices are asking prices, not final sale prices.

2. Data analysis and preprocessing

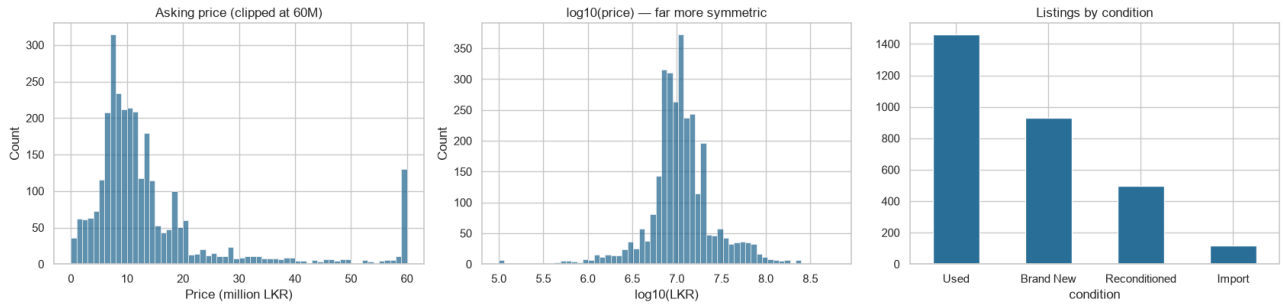


Figure 1 — Price is heavily right-skewed; log-price is near-symmetric.

- Skew: price skewness is extreme (a few LKR 200M+ cars), so models are trained on log(price), which also makes the objective a relative (percentage) error.
- Invalid values, not only nulls: placeholder prices (0, 106, 123456), a 595M-rupee BMW with a 1 cc "electric" engine, years after 2026, engines under 300 cc, mileage placeholders (1, 10, 123456) and "used" cars with under 500 km. 14 rows with impossible prices were dropped; invalid engine/mileage values were set to missing and imputed. Genuine vintage (Sunny, Lancer, Beetle) and luxury cars were kept because they are real parts of the local market.
- Imputation and features: engine size imputed from the median of the same car model; mileage from the median of the same condition; age and km-per-year engineered; rare models and locations grouped as "Other"; ordinal encoding for tree models, one-hot for the linear baseline, native categorical handling in CatBoost, and standard scaling of numeric columns. All steps are fitted on training data only.
- Key EDA findings: condition is the largest price divider (Brand New/Import above Reconditioned above Used); depreciation with age is non-linear; brand and model matter, with 351 distinct car models (a high-cardinality categorical problem); mileage is only moderately informative because it is confounded with age.

3. Models and hyper-parameter design

Two baselines ("existing approaches") — Ridge regression and a single decision tree — were compared with Random Forest, XGBoost, LightGBM, CatBoost and sklearn Gradient Boosting, plus Voting and Stacking ensembles. No default parameters were used: each algorithm received a search space designed from how it works and from the data size (about 2,400 training rows), tuned with 5-fold randomised search. Examples: boosters use shallow trees (depth 3–8) with low learning rates (0.02–0.15) and row/column subsampling to avoid over-fitting a small, noisy dataset; Random Forest uses deep trees (8–30) because bagging reduces variance; LightGBM's num_leaves is constrained to 15–63. The stacking meta-learner (Ridge) is trained on out-of-fold predictions of XGBoost, LightGBM and CatBoost. Random Forest was left out of the ensembles because the meta-learner gave it a weight of only about 0.01 and it made the saved model about 180 MB.

4. Results

Model	R^2 (LKR)	MAPE %	RMSE (log price)	MAE (LKR)
Stacking (XGB+LGBM+Cat → Ridge)	0.914	12.6	0.225	2.00 M
CatBoost	0.905	13.1	0.225	2.13 M
Voting (XGB+LGBM+Cat)	0.914	12.8	0.231	1.94 M
XGBoost	0.906	14.5	0.247	2.16 M
LightGBM	0.901	14.0	0.250	2.12 M
Gradient Boosting (sklearn)	0.893	14.7	0.255	2.26 M
Ridge (baseline)	0.761	16.7	0.272	3.27 M
Random Forest	0.858	15.9	0.273	2.40 M
Decision Tree (baseline)	0.735	23.5	0.335	3.22 M

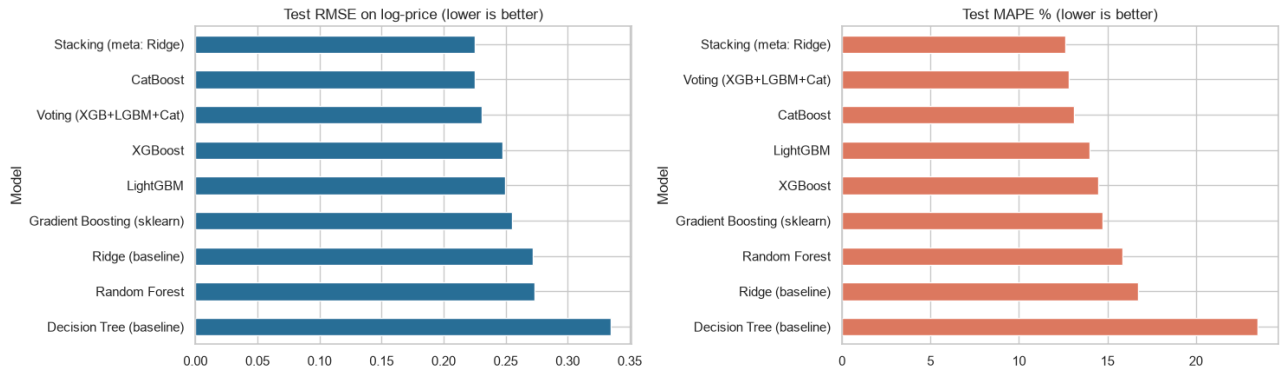


Figure 2 — Test error of every model (lower is better).

- Ensembles clearly beat the baselines: boosting cuts error from 23.5% (single tree) to about 13%, and R^2 in rupees rises from 0.74 to 0.91. Ridge is reasonable on the log scale but poor in rupees (R^2 0.76) because it misprices the expensive tail where brand × age × engine interactions matter.
- CatBoost is the best single model, consistent with the data having many high-cardinality categorical features; the stacker gives it about 64% of the weight (XGBoost 24%, LightGBM 14%).
- Stacking and CatBoost are effectively tied on log-RMSE (0.2249 vs 0.2253); stacking's gain appears in R^2 and rupee error, so it is at least as good but not decisively better, at a higher complexity cost.

- Random Forest under-performed the boosters: it cannot extrapolate beyond the training price range, so luxury cars are under-priced, and it treats integer-coded categories as ordered.
- Cross-validation error (0.242 for stacking) is consistent with test error (0.225), so there is no sign of over-fitting. A typical error of about 13% is close to the natural noise of asking prices.

5. Strengths and limitations of the models

- Ridge — fast and transparent, but cannot capture non-linear depreciation or interactions.
- Decision tree — readable rules but high variance (weakest result).
- Random Forest — robust and low-tuning, but cannot extrapolate and produces large models.
- XGBoost / LightGBM — strong accuracy with regularisation and fast training; need careful tuning, and categoricals must be encoded by hand.
- CatBoost — native categorical handling and ordered boosting give the best single model; slowest to train.
- Voting / Stacking — combine different error patterns for the best accuracy; most complex and costly to explain.

6. How the chosen algorithms differ from decision trees, logistic regression and k-NN

The best models in this project are gradient-boosted tree ensembles (XGBoost, LightGBM, CatBoost) combined by stacking. They differ from the three standard models in several ways. Two notes on scope: logistic regression is a classifier, so for this regression task its counterpart, regularised linear regression (Ridge), was used as the baseline; and k-NN was not trained as a model here, so it is compared by its properties only, not by a measured score.

Aspect	Decision tree	Logistic / linear (Ridge)	k-NN	Boosting + stacking (this project)
How it learns	One set of if/then splits, chosen greedily	One weighted sum of the features	No training: stores all cars and averages the k closest	Hundreds of small trees added one by one, each fitted to the errors of the previous ones; stacking learns how much to trust each booster
Non-linearity and interactions	Yes, but only one tree	No: a straight-line effect per feature	Yes, but only locally	Yes, built up gradually (age x mileage x engine x condition)
Over-fitting	High variance: small data changes give a different tree	Low variance but high bias (underfits depreciation)	Sensitive to k and to noisy features	Controlled by shallow trees, low learning rate, subsampling and regularisation, tuned by 5-fold CV
Many categories (351 car models, 166 locations)	Needs encoding; rare groups give unstable splits	Needs one-hot encoding: very wide	Distance between two categories is meaningless	CatBoost handles categories natively (ordered target statistics); rare ones are grouped
Feature scaling	Not needed	Needed	Needed: mileage in km would dominate the distance	Not needed
Result on this data (test set)	MAPE 23.5%, R2 0.735	MAPE 16.7%, R2 0.761	Not measured	MAPE 12.6%, R2 0.914 (stacking)

Why this matters for car prices: depreciation is non-linear and depends on interactions (a 2 000 cc engine is worth more in a new car than in an old one), the data has many categorical values and only about 2,400 training rows, and the target is noisy. A single tree is unstable here, a linear model is too rigid, and k-NN suffers because cars with the same specification are rare for uncommon models. Boosting reduces both bias and variance, and stacking adds a further small gain by weighting the three

boosters (about 64% CatBoost, 24% XGBoost, 14% LightGBM). The cost is lower interpretability and more tuning, which is why SHAP and LIME (section 7) were added.

7. Explainability: SHAP and LIME

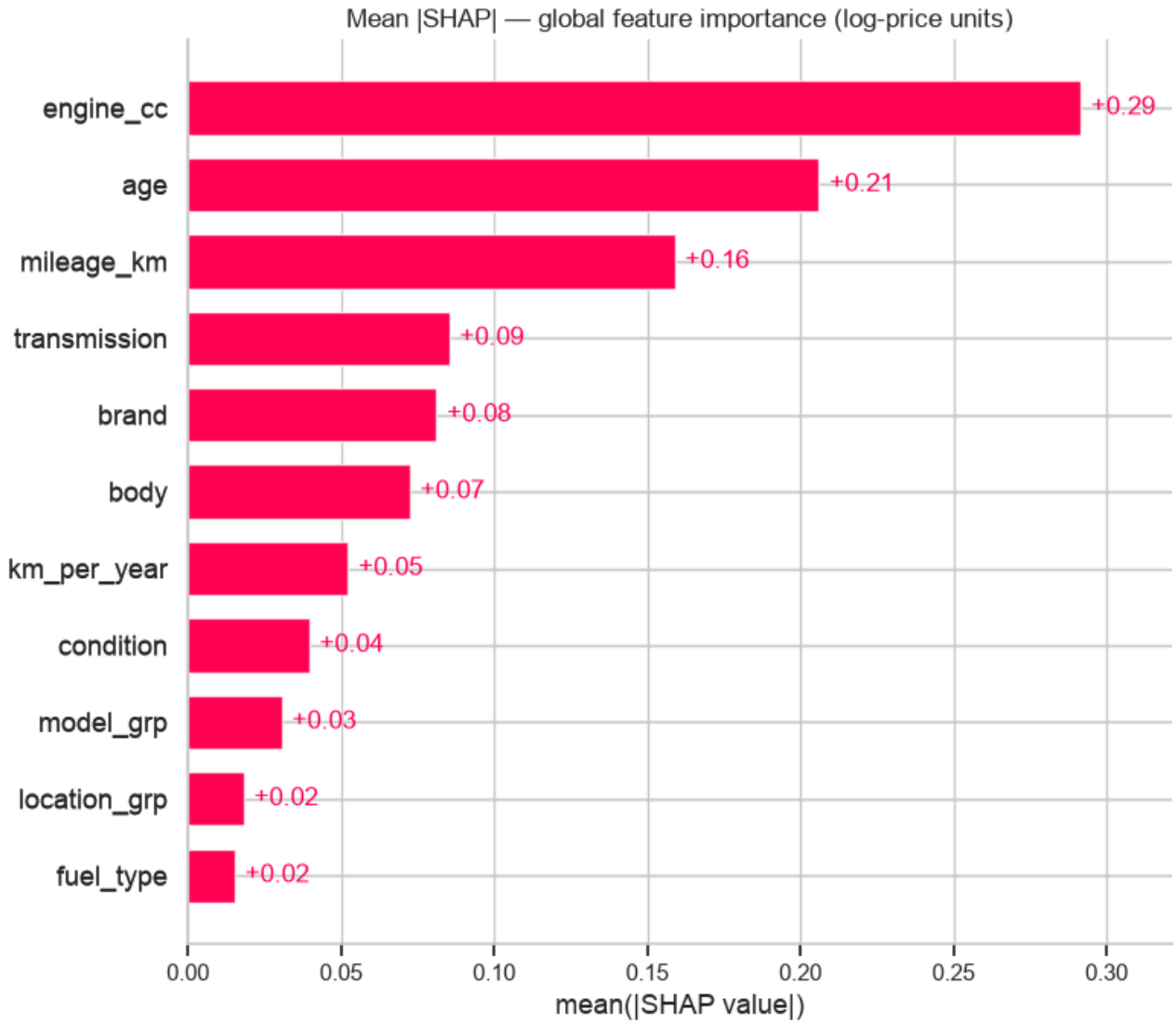


Figure 3 — SHAP global importance (XGBoost), in log-price units.

- SHAP: engine capacity, age and mileage dominate (mean |SHAP| about 0.29, 0.21, 0.16, i.e. roughly 29%, 21% and 16% average price effect). Newer, lower-mileage and larger-engine cars are predicted higher, which matches economic intuition. Engine size is partly a proxy for car class, so SHAP shows what the model uses, not what causes price.
- LIME (local): for a 2018 Suzuki Alto, the small engine and manual gearbox push the predicted price down. For a 2007 Hyundai Santa Fe (over-estimated by about 33%), LIME exposed a recorded mileage of only 0–15,000 km that is implausible for a 19-year-old car — a likely data-entry error — showing that explanations can also act as a data-quality check.

8. Inference and application

One car (a 2026 Daihatsu Mira) was kept out of training, tuning and testing. The exported model (model.pkl, 4 MB) predicted LKR 7.43 M against an actual asking price of LKR 7.48 M (−0.6%). A Streamlit front end (generated with GenAI assistance) loads the pickle, offers drop-downs and number

fields for each input, and returns the estimated price with a ±13% range based on the model's average error.

9. MLOps pipeline (bonus)

To keep the model reliable after deployment, the project includes an automated MLOps loop built on GitHub Actions and Azure.

- CI: every push runs 9 automated tests (cleaning rules, model sanity, a release quality gate of $R^2 \geq 0.85$ and MAPE ≤ 20%, and the app) and builds and health-checks the Docker image.
- Monitoring: weekly, fresh ikman.lk ads (unseen by the model) are compared with the training data using PSI and KS tests, and the live model is scored on them. On 200 fresh ads the live error was MAPE 9.4% with no significant drift.
- Retraining with a gate: on drift (or a forced run) a challenger is trained on old + fresh data. Both challenger and current model are scored on fresh ads neither saw; the challenger is promoted only if it is no worse (in a test: log-RMSE 0.1285 vs 0.1305). A pull request is opened for a human to approve.
- Deployment: the app runs on Streamlit Cloud (redeploys on merge) and on Azure Container Apps (Malaysia West, scale-to-zero, about zero idle cost) from a Docker image published to GitHub Container Registry. Azure app: https://car-price-app.bluemeadow-a6e43a1f.malaysiawest.azurecontainerapps.io
- Limitation: Streamlit Cloud and the free tier keep no persistent prediction log, so fresh ads stand in for production traffic when checking drift.

10. Bias, fairness, real-world impact and ethics

Risk of bias or unfairness

- **Selection bias.** The data covers only ads on ikman.lk, so it reflects sellers who advertise online. The dataset is concentrated in a few areas (Kohuwala alone is 11% of ads, the top five locations about 33%), 137 of the 166 locations have fewer than 30 ads, and Toyota, Suzuki and Honda make up about 58% of the cars. Buyers or sellers outside the Colombo suburbs are therefore less well represented.
- **Unequal accuracy across groups.** Popular cars have many comparables; vintage cars (130 ads before 2000) and expensive cars (about 10% above LKR 30 million) have few. Errors there are larger, for example the 2007 Hyundai Santa Fe was over-estimated by about 33%. A seller of a rare car could be given a less reliable price than a seller of a common one.
- **Location as a proxy.** Location is a model input. It can encode the wealth of an area, not the quality of the car, and rare locations are grouped as "Other", which treats small towns as one block. Seller type (dealer or private) and seller identity are not model features, so the model does not price by who is selling, but location can still correlate with it.
- **Learning seller behaviour.** The target is the asking price. If dealers or some regions habitually ask for more, the model learns and repeats that, and it does not know the true sale price.

Real-world impact

- **Benefits.** A buyer or seller with little market knowledge gets a quick reference price and can spot an ad that is far above or below the usual level, which reduces information asymmetry in a market where import restrictions made prices hard to judge.

- **Harms if misused.** An error of 13% on a LKR 10 million car is about LKR 1.3 million. If the estimate is treated as a fixed valuation it could anchor negotiations unfairly, and it should not be used for loans, insurance or tax. If many sellers price from the same tool, it could also push asking prices together. For this reason the app shows a range, labels the result as an estimate of the asking price, and is described as decision support.
- **Changing market.** Prices depend on import policy, exchange rates and fuel prices, so the model goes stale. The MLOps pipeline (section 9) checks drift every week, and a retrained model is promoted only after passing a quality gate and a human approval.

Ethical use of data

- **Collection.** Only publicly visible adverts were collected, the site's robots.txt was respected, requests were spaced 1.5 seconds apart, and no login or protection was bypassed (a second site, Riyasewana, refused automated access and was not circumvented). The data is used for coursework and the source is cited.
- **Personal data.** The model uses only vehicle attributes, price and general location. Sellers' names and contact details are not used. The free-text description column can contain phone numbers, so it is not a model input and should be removed or anonymised before the dataset is shared more widely.
- **Transparency and oversight.** SHAP and LIME explain predictions, the limits are stated, retraining needs a person to approve, and the application does not log users' inputs.
- **Remaining limits.** No fairness metric across regions or seller groups was measured, because the data has no seller attributes beyond a membership flag. A future version should report error by price band and by area and collect more rural and luxury listings.

11. Conclusion, limitations and future work

Tuned boosted-tree ensembles predict Sri Lankan car prices to within about 13% on average, far better than linear or single-tree baselines, and SHAP/LIME make the predictions explainable. Limitations: asking prices rather than sale prices; about 3,000 ads from one site at one time; no trim, photos or accident history; few comparables for vintage and supercars. Future work: collect data over time and add a time-series component for market drift, add Riyasewana data, use quantile regression for proper price intervals, and deploy with an MLOps pipeline (CI/CD and drift monitoring) for the bonus marks.

Data source: ikman.lk classified advertisements, collected for coursework purposes.